National University of Singapore
School of Computing

Evaluating Direct and Solver-Assisted AI Degree Planning

Schedule Validity, Translation Fidelity and Explanation Accuracy

CS3263 Foundations of Artificial Intelligence
Semester 1, AY2026/27

Team member	Matriculation number	Email
Lu Ruijian	A0287930U	e1297772@u.nus.edu
Brandon Chong Wee Kiat	A0269806N	e1114718@u.nus.edu
Jonathan Graziano Yang	A0300413A	e1354779@u.nus.edu

Report draft for team review
14 October 2026

Abstract

We compare direct language-model planning with translation into a constraint satisfaction problem (CSP) followed by symbolic solving. The task concerns the remaining Programme Requirements of an NUS Computer Science student pursuing the Artificial Intelligence focus area, using a restricted catalogue and a 16-unit semester workload cap. A frozen benchmark contains 120 evaluation cases: 84 feasible and 36 infeasible, distributed across four study horizons and three designed difficulty levels. Direct planning achieved 119/120 correct outcomes; solver assistance achieved 120/120. Mean recorded process plus solver time was 22.41 and 31.60 seconds respectively. All five noncanonical translations were reviewed as semantically equivalent. A secondary audit identified 17 incorrect numeric claims in 15 otherwise valid direct-plan responses. The results demonstrate the value of evaluating schedule validity, translation fidelity and explanation accuracy separately. They provide limited evidence for a general accuracy advantage from solver assistance.

1 Introduction and research question

Degree planning coordinates course selection, prerequisite order, recurring offerings and workload. A course combination may meet aggregate requirements while remaining impossible to schedule; a plausible schedule may omit requirements or miscount overlapping groups. These interactions motivate a constraint satisfaction model for an educational assistant.

The project asks: *under the tested conditions, how reliably does a frontier language model plan directly, and what changes when it translates the request for a formal solver?* We examine three proposal sub-problems: selecting courses that satisfy requirements, respecting prerequisite relationships, and assigning courses to semesters subject to workload limits. Soft preferences and optimization are deferred. Every solution that satisfies the specified hard constraints is acceptable.

We contribute an explicit Boolean CSP, a benchmark with validated outcomes, and a paired evaluation separating schedule validity, translation fidelity and explanation accuracy. Primary scores approach the benchmark ceiling; the clearest additional finding is inaccurate numerical explanations accompanying valid plans.

2 Domain and scope

The domain is the AY2026/27 NUS BComp(Computer Science) admission cohort and AI focus area. Official programme and focus-area pages provide the requirements; NUSMods supplies course records [1, 2, 3]. Source snapshots were collected on 9 October 2026. The normalized catalogue contains 87 courses: nine foundations, three mathematics courses, 19 AI breadth courses, one credited internship and 55 prerequisite-support courses. Four courses are non-selectable under the frozen assumptions.

The model covers the 80-unit Programme Requirements, excluding Common Curriculum, Unrestricted Electives, GPA, exemptions, exchange mappings, dissertation substitutions, special terms, quotas and timetable clashes. Prerequisite-support courses consume workload without earning Programme credits. Benchmark requests explicitly require CS1101S although its credits are outside Programme groups.

The workload limit of 16 modelled units per normal semester is a project policy, reserving capacity for other study; it is not an official NUS maximum. Empty semesters are permitted. Normal Semesters 1 and 2 alternate, and the AY2026/27 offering pattern repeats in simulated future years. Standard mathematics, English and programme-entry conditions are assumed satisfied;

passed courses satisfy modelled grade thresholds. CP3200 contributes six completed internship units in every case, including early-stage profiles. Internship eligibility is not simulated. These choices define a simplified remaining-requirements planner rather than a complete graduation audit.

3 Formal CSP formulation

3.1 Inputs, variables and completed credit

Let C be the frozen catalogue, $T = \{1, \dots, H\}$ the remaining semester positions, and $a_t \in \{1, 2\}$ the normal-semester number at position t. For course c, let u_c denote its units, A_c its offering set and $q_c \in \{0, 1\}$ its new-enrolment eligibility. Let $E \subseteq C$ contain completed courses, with $d_c = \mathbf{1}[c \in E]$. Request inputs specify required courses R, forbidden new enrolments F, fixed placements $J \subseteq C \times T$, workload cap W, and requirement groups.

The CSP is $\mathcal{P} = (V, D, K)$, with finite Boolean domains and hard constraints K. Its principal variables are

$$x_{c,t} \in \{0, 1\}, \qquad y_c \in \{0, 1\}.$$

Here $x_{c,t} = 1$ denotes new enrolment at remaining position t; $y_c = 1$ denotes a course completed previously or selected in the plan. Enforce

$$\sum_{t \in T} x_{c,t} \leq 1, \tag{1}$$

$$\sum_{t \in T} x_{c,t} = 0 \qquad \text{if } d_c = 1 \text{ or } q_c = 0, \tag{2}$$

$$y_c = d_c + \sum_{t \in T} x_{c,t}. \tag{3}$$

Thus a completed required course satisfies $y_c = 1$ without being enrolled again. A non-selectable course may still contribute trusted completed credit. Forbidden courses retain past credit but cannot be newly taken.

3.2 Offerings, prerequisites and co-requisites

Offering restrictions impose $x_{c,t} = 0$ whenever $a_t \notin A_c$. For a prerequisite course p, define completion before t by

$$b_{p,t} = d_p + \sum_{s<t} x_{p,s}.$$

A prerequisite taken concurrently is insufficient. Missing catalogue leaves evaluate to false, so an OR expression can use another available alternative. Normalized trees contain course leaves, constants, AND, OR and k-of operators. A Boolean auxiliary variable represents each compound expression exactly: for children $v_1, \dots, v_n$,

$$z = 1 \iff \sum_{i=1}^{n} v_i \geq k.$$

AND uses $k = n$ and OR uses $k = 1$. Both directions of the equivalence are encoded. If $P_c(t)$ is the resulting prerequisite truth value, enforce $x_{c,t} \leq P_c(t)$. Unselected courses do not force their prerequisite chains.

For an ordinary co-requisite p of c, require $x_{c,t} \leq b_{p,t} + x_{p,t}$. The explicitly modelled CS2101/CS2103T pair has the stronger new-enrolment rule

$$x_{\text{CS2101},t} = x_{\text{CS2103T},t}.$$

Historical completed records are trusted. The model does not retrospectively audit a supplied transcript.

3.3 Preclusions and request constraints

For a precluded pair (c, p), impose $y_c + y_p \leq 1$ when at least one course is not already completed. A listed preclusion in either direction constrains the pair. The exception for two completed courses preserves the policy of trusting historical credit. Required, forbidden and fixed courses impose, respectively,

$$y_c = 1 \ (c \in R), \qquad \sum_t x_{c,t} = 0 \ (c \in F), \qquad x_{c,t} = 1 \ ((c,t) \in J).$$

A fixed position refers to the remaining-semester index, not to the normal semester number. For example, positions in $[2, 1, 2]$ correspond to normal Semesters 2, 1 and 2.

3.4 Workload and requirement groups

Semester workload includes every newly enrolled catalogue course:

$$\sum_{c \in C} u_c x_{c,t} \leq W, \qquad W = 16.$$

For a requirement group g with membership M_g, define

$$U_g = \sum_{c \in M_g} u_c y_c, \qquad N_g = \sum_{c \in M_g} y_c.$$

Each specified unit or course-count bound constrains U_g or N_g. Unspecified bounds add no constraint. A course counts once within a group; membership in several groups can satisfy several conditions without creating extra earned credits.

Table 1: Requirement groups in the frozen model.

Group	Membership and bound
Breadth	AI breadth courses and industry credit; at least 32 units.
Upper breadth	Breadth members at level 4000 or above; at least 12 units.
AI primary	Available AI primaries, including foundation CS2109S; at least three courses.
AI upper	AI primaries at level 4000 or above; at least one course.
Industry	CP3200 in the selected pathway; 6–12 units.
Other CP	Non-industry CP breadth courses; at most 12 units. This set is empty in the catalogue.
Programme lower	Programme members at levels 1000–4000; at least 64 units, derived from the fixed 80-unit target.

Foundation and mathematics courses are individually mandatory. Their credits do not also count toward breadth. CS2109S can satisfy an AI focus count while remaining excluded from breadth credits. This distinction is important both for formal constraints and for explanations of a valid plan.

3.5 Solver implementation and independent checking

The implementation uses Python 3.11.9 and OR-Tools 9.15.6755. The Boolean CSP is compiled into CP-SAT with no objective. Unit coefficients are multiplied by 100 to obtain integers; values that cannot be represented in nonnegative hundredths are rejected. CP-SAT supports feasibility solving and distinguishes proven infeasibility from an unknown result [4]. We use one search worker, random seed 3263 and a ten-second evaluation solver limit. Backend feasible or optimal statuses are returned as feasible assignments; an unknown status is never relabelled infeasible.

An independently implemented arithmetic and set-based checker assesses every feasible output against the *original* specification. It checks schema, semester count, duplicates, eligibility, offerings, prerequisite order, co-requisites, preclusions, workload, request constraints and group totals. It does not invoke the solver to validate a schedule. The checker and solver share the normalized catalogue and interpretations, so this independence does not eliminate errors in shared source assumptions.

The pre-evaluation freeze records 25 passing implementation and protocol tests. They cover real-course examples, workload and prerequisite violations, completed credit, OR alternatives, preclusions, fractional credits, group accounting, and rejection of changed or historical experiment inputs. These checks support implementation consistency rather than certifying every institutional rule.

4 Benchmark construction

4.1 Study stages and sampling

The benchmark contains 24 development cases and 120 evaluation cases. Four horizons, $H \in \{7, 6, 5, 4\}$, are crossed with three designed difficulty levels. Each of the 12 cells contains one feasible and one infeasible development case, plus seven feasible and three infeasible evaluation cases. Evaluation therefore includes 84 feasible and 36 infeasible requests.

Table 2: Study-stage profiles used to construct the benchmark.

Remaining semesters	Past semesters	First normal semester	Academic courses passed
7	1	2	2–4
6	2	1	5–7
5	3	2	8–9
4	4	1	10–12

Synthetic academic histories satisfy earlier prerequisites, offerings, co-requisites and past workload limits. CP3200 credit is added separately. The development and evaluation splits have disjoint completed-course sets. Eight distinct histories are used in development and 70 in evaluation. Histories recur within each split, and both splits use related request templates and the same catalogue. These are correlated controlled cases, not independent student samples or a template-disjoint generalization test.

Table 3: Designed restrictions in feasible requests.

Difficulty	Requested electives	Fixed placements	Forbidden alternatives
Easy	1	0	0
Medium	3	2	2
Hard	6	5	4

The catalogue, programme bounds and workload cap remain fixed. References are any satisfying assignments; extra permitted courses and empty semesters are valid. Fixed placements are selected from a feasible reference and forbidden alternatives from unused courses, followed by revalidation. The labels describe construction choices, not measured proof difficulty. More fixed placements may reduce assignment freedom.

4.2 Feasible references and negative cases

Every feasible reference passes the independent checker. Infeasible cases are derived from a checked feasible precursor and carry a concrete contradiction. The nine annotated mechanisms are: required-but-forbidden courses; unavailable offerings; prerequisites impossible in the first remaining semester; preclusions; split simultaneous enrolments; prerequisite ordering; co-requisite workload overload; insufficient lower-level credits; and exhaustion of prerequisite alternatives. Each mechanism occurs in four evaluation cases. Solver-unknown requests are not accepted as infeasible labels.

A witness is a sufficient contradiction rather than a claim about the sole cause. For example, an overload request can also place an advanced course before its prerequisites. This limits conclusions about reasoning difficulty by mechanism. A valid infeasibility explanation can identify another sufficient contradiction.

Evaluation cases were selected without language-model answers. A 12-case paired development pilot achieved 12/12 for both approaches and informed inspection and scoring; its scores are excluded from evaluation totals. A project participant reported source/assumption review complete on 11 October 2026. Reference-label validation was automated; no separate case-by-case human label audit is claimed.

5 Evaluation methodology

5.1 Paired approaches and information supplied

Both approaches receive the same natural-language request and frozen domain facts. The context includes normalized prerequisites, offerings, preclusions, eligibility, and an enumerated narrative of requirement-group membership and bounds. It excludes gold specifications, reference plans and labels. Consequently, the translation task is largely extraction into a fixed contract rather than interpretation of unprocessed university policy pages.

In *direct planning*, the model returns a plan JSON object with status, one course list per remaining semester when feasible, and a brief explanation. It may return infeasible or unknown. In *solver-assisted planning*, the model returns a specification containing completed courses, alternating normal-semester numbers, workload cap, required and forbidden courses, fixed placements, and requirement groups. A fixed compiler independently enforces catalogue course constraints. The model generates no executable solver code. The evaluator then runs CP-SAT and checks a feasible schedule against the original specification.

The task-specific instructions differ. In particular, the translation prompt explicitly states that completed mandatory courses remain required; the direct prompt prohibits re-enrolment without equally explicit guidance on the meaning of requested inclusion for a passed course. This distinction is a potential confound in attributing any observed advantage to the solver.

5.2 Execution controls and provenance

The recorded model identifier is `gpt-6.1-sol`, with medium reasoning effort, accessed through subscription-authenticated Codex CLI `0.162.0-alpha.17.2`. Each case and approach receives

one fresh ephemeral session, a schema-constrained response, and a 180-second process timeout. Web access, shell execution, apps and other tool capabilities are disabled, and event records are audited. Cases run in fixed order, direct followed by assisted. There are no response repairs, retries or feedback from reference outcomes.

The run `csp16-evaluation-v1` was frozen before evaluation. Its manifest records configuration, case order, scoring policy and hashes; a 426-file snapshot preserves inputs and implementation. Completion checks verified all 240 prompt hashes, frozen files, documentation and pilot evidence. Results and raw responses are preserved at commit `d251777` [5].

5.3 Scoring and secondary analysis

The primary policy, `outcome-v1-explanation-secondary`, accepts a schema-valid feasible response only when its plan passes the original-specification checker and the reference case is feasible. An infeasible prediction is correct when it agrees with a validated infeasible label. Unknown outputs, execution failures and malformed responses count as unsuccessful completed attempts. Accuracy uses completed selected attempts, with pending attempts reported separately.

Explanations do not change a valid schedule's primary score. They are reviewed separately. Translation auditing first compares unordered field values and named groups with the reference specification, then reviews differences as equivalent, non-equivalent or unresolved. A matching solver outcome alone is insufficient evidence of faithful translation.

Runtime is recorded process time plus solver time for the assisted approach; shared post-hoc checking and manual review are excluded. CLI token counts are descriptive usage records, not dollar-cost estimates. The later numeric explanation audit is diagnostic and covers explicit matched claim patterns. It was not a separately preregistered comparative endpoint.

6 Results

6.1 Primary correctness and execution

All 240 attempts completed with execution status `ok`, and no tool invocations were recorded. Both approaches correctly identified all 36 infeasible cases. Every returned feasible schedule passed the original specification checker. The direct approach instead declared one feasible case infeasible, producing the only primary failure.

Table 4: Primary outcomes on the 120 evaluation cases.

Measure	Direct	Solver-assisted
Valid plans on feasible cases	83/84	84/84
Correct infeasibility judgments	36/36	36/36
False infeasibility judgments	1	0
Correct outcomes overall	119/120	120/120
Overall accuracy	99.17%	100.00%
Both-correct paired cases	119/120	
Assisted-only correct paired cases	1/120	

Table 5: Correct outcomes by horizon and designed difficulty. Each cell has seven feasible and three infeasible cases.

Remaining semesters	Difficulty	Direct	Solver-assisted
7	Easy	10/10	10/10
7	Medium	10/10	10/10
7	Hard	10/10	10/10
6	Easy	10/10	10/10
6	Medium	10/10	10/10
6	Hard	10/10	10/10
5	Easy	10/10	10/10
5	Medium	10/10	10/10
5	Hard	10/10	10/10
4	Easy	9/10	10/10
4	Medium	10/10	10/10
4	Hard	10/10	10/10

The single failure occurs in an easy cell, so these scores do not exhibit an accuracy decline with designed difficulty or longer horizons. One discordant pair gives a descriptive difference of 0.83 percentage points. It does not establish a broad population-level advantage, particularly given correlated cases, one attempt per case and different prompt guidance.

6.2 Runtime and reported usage

Mean recorded time was 22.41 seconds for direct planning and 31.60 seconds for solver assistance: 9.19 seconds, or approximately 41.0%, longer. Medians calculated from the case records were 24.18 and 31.00 seconds. Mean local assisted solver time, including compilation, was approximately 0.105 seconds. Most recorded assisted latency therefore occurred in the model process rather than local solving.

Reported input-token totals were 2,332,737 and 2,359,601; output-token totals were 62,736 and 133,277 for direct and assisted execution respectively. The larger specification response is consistent with greater serialization work, but token counts do not isolate the cause of latency. Execution order, network and service conditions were not randomized or controlled.

7 Error analysis and translation fidelity

7.1 The false infeasibility case

In `test-h4-easy-01`, the student has already passed CS1101S and asks that the plan include it and CS4277. The direct response treats CS1101S as a compulsory new enrolment and declares a contradiction with the no-retake rule. The formal interpretation distinguishes $y_{\mathrm{CS1101S}} = 1$ from $x_{\mathrm{CS1101S},t} = 1$: past completion satisfies the requirement, and there is no fixed new placement for this course. A feasible reference schedule independently passes all checks. This is an interpretation error under the frozen semantics. The prompt difference discussed above prevents attributing it solely to a lack of symbolic search.

7.2 Canonical differences and equivalent specifications

Automatic field comparison found 115/120 canonical translations. All five remaining specifications were reviewed as equivalent under the frozen catalogue and compiler semantics. Table 6

summarizes the case-specific reasons; each concerns constraints in both directions. These classifications are recorded separately from the unchanged automated diagnostic fields [6].

Table 6: Semantic review of the five noncanonical translations. Case identifiers omit the common `test-` prefix.

Case(s)	Constraint-level equivalence evidence
`h6-easy-07`	A new group requires all 14 courses already individually required, repeating the same conditions.
`h4-easy-03`	
`h4-easy-07`	Four added forbidden courses are already non-selectable; CP3200 past credit remains counted.
`h4-hard-01`	The 19 individual requirements are replaced by a group containing exactly those 19 members and requiring all of them.
`h4-hard-08`	Added required CS4278 is already forced by its original fixed placement in position 1.

Outcome agreement alone would not support these judgments: a changed specification can coincidentally produce a valid plan or share an infeasible status. Here the added restrictions are redundant or the representation is logically equivalent. Canonical accuracy of 95.83% therefore understates the reviewed semantic fidelity of these particular outputs.

7.3 Numeric explanations of valid direct plans

A diagnostic audit checked 453 explicitly matched numerical claims across all 83 feasible direct responses. Literal-claim review and independent checker evidence confirmed 17 incorrect claims in 15 responses (18.1%). Errors concern focus-course counts, credit totals and a semester load. The affected plans still satisfy all original constraints.

Table 7: Examples of explanation errors in independently valid plans.

Case	Claim type	Reported	Checked
`h4-easy-04`	AI primary / AI upper counts	8 / 4	7 / 3
`h4-hard-02`	Final-semester workload	16 units	12 units
`h6-hard-03`	Programme-lower credits	70 units	74 units

The audit is not exhaustive: only defined numeric patterns were checked, so 18.1% is the observed proportion with detected errors, not an estimate of all possible prose inaccuracies. No primary score was changed. Assisted outputs use a generic solver status explanation; this study does not compare two equally detailed natural-language explanation systems.

8 Discussion and threats to validity

8.1 What the comparison establishes

Under a restricted catalogue, explicit domain facts and a fixed output contract, direct planning was already highly accurate. The assisted pipeline provided a satisfying assignment or a solver infeasibility result for every translated instance, and every feasible assisted plan passed the original checker. Its measurable disadvantages in this run were additional latency and larger reported output-token usage. Its architectural benefit is an explicit representation that can

be inspected, solved and checked. This benefit is conditional on correct interpretation and compilation.

The clearest additional finding is that correctness of a schedule does not guarantee correctness of its explanation. Independent arithmetic checking made it possible to detect inaccuracies without discarding valid schedules. A practical assistant could derive counts and credit summaries from validated schedules, then constrain narrative explanations to those values. This is a design implication of the findings; that improved explanation pipeline has not been evaluated here.

Semantic translation assessment likewise requires more than literal comparison. Our five reviewed differences show concrete equivalent representations. They do not prove that field mismatches are generally harmless or that agreement with a reference label establishes fidelity. Both schedule and translation evaluation remain necessary because an infeasible solver result for a mistranslated request could wrongly reject a feasible student request.

8.2 Representation size and designed difficulty

With 87 courses and H remaining semesters, the main model contains $87H$ enrolment variables and 87 completion/selection variables before logical auxiliaries. Ignoring other constraints, at-most-once enrolment permits at most $(H+1)^{87}$ course-position combinations; completed credit, eligibility and offerings substantially reduce this space. Saved validation examples contain 2,160 backend variables and 3,516 constraints for seven semesters, and 1,248 variables and 2,127 constraints for four semesters. These are representation measurements of two examples, not counts of explored states or evaluation-case averages.

Large formal representation size did not translate into observed answer failures on the designed hard cells. Many negative cases contain a short sufficient contradiction, while fixed placements constrain choices. The benchmark therefore probes specified constraint interactions without locating a general difficulty threshold at which direct planning fails. No search-state enumeration, optimization comparison or component ablation was performed.

8.3 Limits of the evidence

First, feasible cases are retained after solver validation and some restrictions are selected from reference plans. This provides consistent labels but does not sample all possible student requests. The 70 evaluation histories and repeated templates create dependence among cases. Disjoint completed-course sets avoid one form of overlap with development, while shared construction limits broader generalization.

Second, the domain context enumerates group memberships and supplies normalized course facts. The model is not required to retrieve current policies or resolve every natural-language institutional exception. Full graduation planning, uncertainty about future offerings and unconstrained student preferences remain outside the experiment.

Third, only one model setting and one attempt per case were tested. Near-ceiling accuracy and one discordant pair offer little evidence about repeated-run stability. Prompt asymmetry and fixed execution order further limit causal claims about solver use and runtime.

Finally, the solver and checker can share an incorrect source interpretation. The semantic and numeric audits were supported by Codex and recorded with case-level evidence. They should not be represented as an independent human replication. Further student review improves confidence in the report, but additional experiments are not assumed in the stated findings.

9 Responsible AI considerations

False infeasibility can reject a valid study route, while inaccurate explanations can create misleading confidence in a valid schedule. A deployment should show checked constraint evidence, distinguish unknown from infeasible, and route disputed interpretations to university guidance.

The benchmark uses synthetic histories and public course information. Deployment with personal transcripts would require consent, access controls, data minimization and a policy for missing or disputed completed credit.

Standard-entry assumptions and internship credit exclude some pathways; performance for bridging requirements, exemptions, exchange, double degrees or different internship opportunities is untested. These inputs require explicit handling. The workload cap and repeated offerings are modelling choices, and feasibility does not certify graduation eligibility or future course availability.

Retained source snapshots, interpretations, responses, hashes and independent checks support traceability. Displayed totals should come from checked evidence, and interpreted constraints should remain inspectable. AI assistance is disclosed because Codex supported both implementation and secondary review.

10 Conclusion

The evaluation found 119/120 correct direct outcomes and 120/120 assisted outcomes, with approximately 41% greater mean recorded time for assistance. Both correctly identified all 36 infeasible cases; every returned feasible schedule passed the original checker. Five noncanonical translations were equivalent, while 15 valid direct responses contained numeric explanation errors.

Both approaches perform strongly on these structured, simplified instances. Primary schedule accuracy conceals a separate explanation-reliability problem. The study supports evaluation that distinguishes schedule validity, translation fidelity and explanation accuracy. It does not establish a broad solver superiority claim or the constraint-complexity boundary at which assistance becomes substantially more beneficial.

11 Team contributions and AI assistance

The following table summarizes the team's division of work.

Team member	Contribution
Brandon Chong Wee Kiat	Formulated the project idea; collected and processed the initial dataset.
Lu Ruijian	Built the benchmark; implemented the solver-assisted pipeline; coordinated and organized the report.
Jonathan Graziano Yang	Conducted the evaluation; contributed to report writing.

Codex supported source acquisition and normalization, CSP and checker implementation, benchmark generation, execution tooling, test construction, experiment execution, secondary analysis and preparation of this report draft. Student participants selected the scope and modelling policies, directed revisions and reported completion of the source/assumption review. The team remains responsible for understanding and verifying the formulation, conclusions and contribution statement before submission.

References

[1] NUS School of Computing. *CS Cohort 2026/2027: Degree Requirements.* `https://www.comp.nus.edu.sg/cug/per-cohort/cs/cs-26-27/`. Experiment source snapshot collected 9 October 2026.

[2] NUS School of Computing. *CS Focus Areas: Artificial Intelligence.* `https://www.comp.nus.edu.sg/programmes/ug/focus/`. Experiment source snapshot collected 9 October 2026.

[3] NUSMods. *Public API, academic year 2026–2027.* `https://api.nusmods.com/v2/`. Per-course endpoint: `/2026-2027/modules/COURSE.json`. Experiment source snapshots collected 9 October 2026. Project and API provenance: `https://github.com/nusmodifications/nusmods`.

[4] Google OR-Tools. *CP-SAT Solver.* `https://developers.google.com/optimization/cp/cp_solver`. Accessed 14 October 2026. Experiment software version: 9.15.6755.

[5] CS3263 project team. *AI Degree Planning Assistant: frozen implementation and paired evaluation.* Repository commit `d2517771fd28edf657475a4ea1f68096458e035b`. `https://github.com/RuijianLu/CS3263Project/tree/d2517771fd28edf657475a4ea1f68096458e035b`. Primary evidence: `results/csp16-evaluation-v1/metrics.json`, `summary.csv` and `completion.json`; freeze and raw records: `results/runs/csp16-evaluation-v1/`.

[6] CS3263 project team, with Codex assistance. *Secondary audit of csp16-evaluation-v1.* `https://github.com/RuijianLu/CS3263Project/blob/d2517771fd28edf657475a4ea1f68096458e035b/results/csp16-evaluation-v1/manual_audit.json`. Numeric-claim evidence is preserved in `results/csp16-evaluation-v1/explanation_numeric_checks.json` at the same commit.